Engineering Applications of Artificial Intelligence 181 (2026) 115443

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Research paper

SHIELD: Advanced persistent threats detection and intelligent explanation using large language models

Parth Atulbhai Gandhi, Prasanna N. Wudali, Yonatan Amaru, Akansha Shukla, Yuval Elovici, Asaf Shabtai *

Ben-Gurion University of the Negev, Israel

ARTICLE INFO

Keywords:
Large language model
Provenance graph
Advanced persistent threat detection
Local outlier factor
Louvain community detection
Natural-language attack narrative

ABSTRACT

Advanced persistent threats (APTs) are sophisticated cyber attacks that can remain undetected for extended periods, making their mitigation particularly challenging. Given their persistence, significant effort is required to detect them and respond effectively. Existing provenance-based attack detection methods often lack interpretability and suffer from high false positive rates, while investigation approaches are either supervised or limited to known attacks. To address these challenges of threat detection and investigation, we introduce SHIELD, a novel approach that combines statistical anomaly detection and graph-based analysis with the contextual analysis capabilities of large language models (LLMs). SHIELD leverages the implicit knowledge of LLMs to uncover hidden attack patterns in provenance data, while reducing false positives and providing clear, interpretable attack descriptions. This reduces analysts' alert fatigue and makes it easier for them to understand the threat landscape. Our extensive evaluation demonstrates SHIELD's effectiveness and computational efficiency in real-world scenarios. SHIELD was shown to outperform state-of-the-art methods, achieving higher precision and recall. SHIELD's integration of anomaly detection, LLM-driven contextual analysis, and advanced graph-based correlation establishes a new benchmark for APT detection.

1. Introduction

Advanced persistent threats (APTs) are becoming a growing concern for today's organizations due to their sophistication and stealth. The 2020 SolarWinds supply chain attack (Lazarovitz, 2021) in which attackers remained undetected for over nine months while compromising 18,000 organizations and major U.S. government agencies demonstrates the harm APTs can cause. Another APT, APT-C-36 (Blind Eagle),[1] has been actively targeting the financial and government sectors in South America since 2018, employing advanced social engineering and fileless malware techniques to evade detection while maintaining a long-term presence in compromised networks. These incidents, which show how APTs can persist undetected for long periods of time, highlight the urgent need for sophisticated detection approaches that can identify threats without overwhelming security analysts with false positives.

The analysis of system provenance data, which captures detailed relationships and interactions between processes, files, and network connections over time, has emerged as a promising approach for APT detection (Zipperle et al., 2022; Inam et al., 2023). By maintaining a comprehensive record of system activities and their dependencies, provenance data can expose subtle patterns and lateral movements characteristic of sophisticated attacks. Existing detection methods relied on expert-defined rules to identify known attack patterns in provenance data (Hossain et al., 2017; Milajerdi et al., 2019; Hassan et al., 2020; Kurniawan et al., 2022). While effective against known threats, these rule-based approaches require constant updates and struggle with zero-day attacks. To address these limitations, researchers shifted toward machine learning-based anomaly detection methods that model normal system behavior to identify deviations. These approaches range from statistical analysis (Hossain et al., 2017; Hassan et al., 2019a; Wang et al., 2020; Kurniawan et al., 2022; Dong et al., 2023b; Li et al., 2024) to deep learning models using path (Du et al., 2017; Zhang et al., 2019; Guo et al., 2021; Alsaheel et al., 2021) and graph-based architectures (Han et al., 2020; Wang et al., 2022; Zengy et al., 2022; Jia et al., 2023; Yang et al., 2023) for pattern recognition.

However, current machine learning (ML) methods face two significant limitations. These methods require extensive training data and struggle to adapt to the dynamic nature of both system behaviors and

* Corresponding author.
E-mail addresses: gandhip@post.bgu.ac.il (P.A. Gandhi), wudalipr@bgu.ac.il (P.N. Wudali), amaruy@post.bgu.ac.il (Y. Amaru), akansha@post.bgu.ac.il (A. Shukla), elovici@bgu.ac.il (Y. Elovici), shabtaia@bgu.ac.il (A. Shabtai).

[1] https://www.trendmicro.com/en_us/research/21/i/apt-c-36-updates-its-long-term-spam-campaign-against-south-ameri.html

https://doi.org/10.1016/j.engappai.2026.115443
Received 10 April 2025; Received in revised form 5 February 2026; Accepted 11 June 2026
Available online 23 June 2026
0952-1976/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

evolving attack patterns, leading to a high rate of false positives. In addition, existing approaches for interpreting alerts rely on labeled training data, which are tedious to generate and quickly become outdated (Alsaheel et al., 2021; Milajerdi et al., 2019; Hossain et al., 2017; Hassan et al., 2020), or focus on ML explainability rather than the specific characteristics of attacks.

Recent advances in large language models (LLMs) have introduced new capabilities for understanding and reasoning about complex system behaviors. These models, which have a remarkable ability to adapt to different contexts and provide logical explanations for their findings, could serve as the basis of new APT detection approaches, overcoming the shortcomings of existing approaches. However, LLMs also have two critical limitations. First, directly feeding raw system logs to LLMs is impractical and inefficient, requiring to provide them with pre-processed logs using traditional security analysis methods like anomaly detection. Second, LLMs are prone to hallucination, which can be addressed through multiple complementary techniques like prompt engineering, chain-of-thought reasoning, and validation checks.

Building on these mechanisms, we propose SHIELD to address the challenges of uninterpretable attack graphs for analysis, concept drift, and lack of time-aware probability mechanisms. SHIELD is a novel framework that combines unsupervised anomaly detection with language model reasoning for APT detection and investigation. Specifically, SHIELD performs attack investigation by transforming complex, uninterpretable attack graphs into clear, actionable natural-language reports; by reducing false positives, SHIELD significantly reduces analysts' efforts and improves response time compared to other mainstream methods. To counter concept drift, SHIELD employs lightweight anomaly detection techniques and leverages off-the-shelf LLMs, enabling adaptive threat detection without resource-intensive retraining, and supporting dynamic policy updates through prompt modifications without additional training overhead. Furthermore, SHIELD introduces unique time-aware probability decay and boosting mechanisms, which dynamically adjust the risk scores of suspicious events in a single streaming loop, ensuring the accurate identification of slow-evolving, stealthy attacks while maintaining low false positive rates. Finally, we apply a strict chronological evaluation methodology to prevent data leakage and ensure realistic performance metrics, overcoming the limitations of prior methods.

Our framework accumulates system logs in a sliding window mechanism for analysis, while leveraging a temporal correlation engine to track the historical evolution of suspicious events to enable the detection of attacks spanning over a long period of time. The proposed short-window mechanism is critical for rapid threat identification, as it detects and responds to attacks with minimal latency. SHIELD (1) employs statistical analysis to flag anomalous events; (2) constructs provenance graphs to determine relationships; (3) prunes benign activities; and (4) clusters suspicious events using community detection algorithms. These steps reduce large amounts of system events to concise groups of suspicious activities, preserving significant attack patterns. An LLM then performs multi-stage reasoning on the identified clusters, providing interpretable insights and mapping them to the APT kill chain.

The temporal correlation engine continuously compares new alerts with historical events. It employs dynamic confidence scoring using decay and boosting mechanisms. The decay mechanism lowers the confidence score if flagged processes consistently exhibit benign behavior, while the boosting mechanism increases scores as additional attack stages are detected across time windows. This dual-mechanism approach maintains a comprehensive view of slow-evolving attacks while preserving real-time detection capabilities and ensuring low false positive rates. Unlike existing approaches that focus solely on anomaly or rule-based detection, SHIELD combines these techniques with LLM-based reasoning to bridge the gap between detection and actionable security insights.

We evaluate SHIELD on four diverse datasets: the DARPA Eng. 3 CADETS and THEIA datasets,[2] the Public Arena dataset,[3] and an in-house dataset containing the Blind Eagle APT-C-36 campaign. Using only 30% of each dataset for training, SHIELD achieved perfect precision on the CADETS dataset and maintained high recall (0.93–1.00) across all datasets. SHIELD successfully tracked attacks spanning multiple days, with detailed attack narratives mapping precisely to APT stages from initial compromise through lateral movement and data exfiltration. Most notably, when analyzing the key CADETS attack sequence, SHIELD identified 25 true positive events with zero false positives, while baseline methods generated over 4000 false events requiring analyst investigation.

The key contributions of this work can be summarized as follows: (1) To the best of our knowledge, SHIELD is the first framework to employ LLMs as the primary unsupervised detection mechanism on provenance graph communities, combined with temporal correlation for tracking multi-stage APT attacks. (2) We developed a novel engine that can track attacks spanning over a long time period using dynamic confidence scoring with boosting and decay mechanisms, effectively identifying slow-evolving, stealth attacks. (3) We designed a novel framework capable of generating human-interpretable, comprehensive attack summaries that maps attack events to kill-chain stages. These summaries, enriched with relevant IoCs, enable security teams to quickly analyze and respond to potential threats. (4) We leverage the capabilities of off-the-shelf LLMs, eliminating the need for traditional model training. Unlike deep learning-based methods, that suffer from concept drift, our system maintains adaptability through prompt-based contextualization of organizational changes.

2. Related work

Provenance-based approaches for APT detection have been developed to address increasingly sophisticated cybersecurity challenges. Rule-based techniques (Milajerdi et al., 2019; Yu et al., 2019) utilize predefined security policies and heuristic rules to identify attack patterns. While these methods offer detailed, fine-grained event-level detection (Inam et al., 2023) and typically maintain low false positive rates, they require extensive manual efforts and are less effective in detecting zero-day exploits (Hossain et al., 2020).

Anomaly-based detection methods, including statistical techniques (Hossain et al., 2017; Hassan et al., 2019a; Wang et al., 2020; Kurniawan et al., 2022; Dong et al., 2023b; Li et al., 2024), path-based approaches (Du et al., 2017; Zhang et al., 2019; Guo et al., 2021; Alsaheel et al., 2021), and graph-based methods (Han et al., 2020; Wang et al., 2022; Zengy et al., 2022; Jia et al., 2023; Yang et al., 2023), have also demonstrated promising results in detecting APTs.

Recent advances in provenance-based intrusion detection have enabled the development of sophisticated deep learning approaches that demonstrate improved investigation capabilities but face significant operational challenges (Bilot et al., 2025). ORTHRUS (Jiang et al., 2025) employs a spatio-temporal graph neural network encoder trained on benign historical data to perform node-level anomaly scoring, demonstrating fine-grained detection capabilities. Similarly, FLASH (Rehman et al., 2024) leverages joint semantic-structural embeddings through Word2Vec and GNN architectures to enable real-time tagging and attack-evolution graph generation, providing enhanced attack reconstruction capabilities. However, these graph neural network based methods, often struggle to adapt to changes in system behavior over time, which can lead to high false positive rates overwhelming security analysts. In addition, the alerts generated by these techniques often lack interpretability, making it challenging for security analysts to understand and investigate the anomalies detected. Despite their technical sophistication, these state-of-the-art approaches share a fundamental

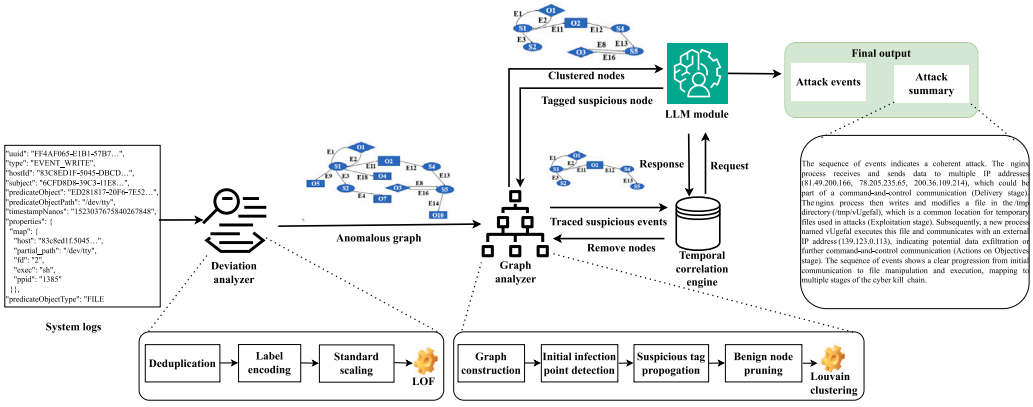


Fig. 1. Overview of the SHIELD pipeline, with an example demonstrating the interaction between the four modules.

limitation: they require extensive retraining when system behaviors evolve or new attack patterns emerge.

Moreover, some of the recent studies (Alsaheel et al., 2021; Wang et al., 2022; Zengy et al., 2022; Jia et al., 2023; Han et al., 2020; Li et al., 2024) have two methodological limitations. First, these methods heavily depend on large training datasets while utilizing disproportionately small testing windows, a practice that conflicts with the dynamic and ever-evolving nature of real-world data environments. Secondly, some studies have identified instances of data leakage (Wang et al., 2022; Jia et al., 2023), where models were unintentionally trained on future data, potentially leading to inflated performance metrics. These limitations highlight the need for additional research in order to develop new approaches to enhance the effectiveness and robustness of these approaches, thereby promoting their wider adoption and practical implementation (van der Kouwe et al., 2019; Dong et al., 2023a).

Other recent research focused on using transformers and LLMs for anomaly detection in logs. For instance, LAnoBERT (Lee et al., 2021) employs regular expressions for minimal log preprocessing, enhancing flexibility across different formats; LogGPT (Han et al., 2023) uses a GPT-2 model with sequential prediction and a top-K reward mechanism to improve detection capabilities; and LogPrompt (Liu et al., 2024) leverages advanced prompt strategies to boost LLM performance in log analysis. While these approaches enable advanced log analysis, research on their application on system-level provenance entities and use in identifying APT attacks remains limited.

Recent work also includes DMMal (Chai et al., 2022), which performs malware detection by enhancing binary-to-image encodings and introducing a sharpness-aware optimizer, resulting in more accurate classification. Building on this foundation, MalFSCIL (Chai et al., 2024) incorporates a variational autoencoder and dynamic class prototypes to enable continual learning, allowing the model to recognize new malware families over time without suffering from catastrophic forgetting. However, both DMMal and MalFSCIL operate primarily at the individual binary level, lacking mechanisms to contextualize those detections within the broader scope of an evolving APT campaign.

More recently, several works have begun exploring LLMs in the context of provenance-based security analysis, though with fundamentally different architectural roles. Similarly, Zuo et al. (2025) employs LLMs for semantic feature enrichment, where the model generates natural language descriptions of system events that are embedded via sentence transformers and fed into supervised or semi-supervised machine learning based classifiers. HuntGPT (Ali and Kostakos, 2023) takes a different approach by applying LLMs for post-hoc explanations of detected anomalies for security analysts.

The main limitations identified in the proposed methods and prior studies can be summarized as follows: (1) rule-based methods require manual efforts and are unable to detect zero-day attacks; (2) anomaly-based methods fail to adapt to evolving system behavior over time, resulting in high false positive rates; (3) alerts generated by anomaly-based methods lack interpretability, making it difficult for security analysts to investigate detected anomalies; (4) in many cases, the evaluation performed used training data that do not reflect real-world scenarios, leading to inflated (incorrect) performance results; (5) existing methods face challenges with scalability and have high computational demands, limiting their practical deployment; and (6) insufficient application of LLMs for the tasks of APT detection and investigation, particularly with the use of provenance data.

In light of these gaps and limitations, we propose SHIELD, a novel framework designed to address the challenges of APT detection and investigation in provenance-based systems.

3. Proposed method

SHIELD consists of four interdependent modules working in a iterative feedback-loop (Fig. 1). The deviation analyzer examines system logs to identify anomalous events. The graph analyzer processes the anomalous events, constructs a graph, prunes benign nodes, and clusters suspicious nodes into communities. The LLM analyzer evaluates these communities, identifying attack patterns and assigning confidence scores. Finally, the temporal correlation engine preserves historical attack data, correlates events across time, and re-initiates analysis when necessary, ensuring continuous and adaptive threat detection.

Deviation Analyzer. The deviation analyzer serves as the foundation of SHIELD's detection capabilities by identifying behavioral deviations in system logs (see example in Fig. 2). It uses a local outlier factor (LOF) model provided with baseline system activities as a reference. This model processes raw logs to produce a filtered graph of anomalous processes and their immediate lineage.

(a) Log structure and notation. The system logs capture detailed process-object interactions in the operating system. Each log entry l_i in our dataset is represented as: $l_i = (p_i, n_i, e_i, o_i, d_i, t_i)$, where p_i represents the process identifiers, n_i the process name, e_i the event type, o_i the object identifiers, d_i the object data, and t_i the timestamp indicating time at which the event occurred. To efficiently process these logs, deviation analyzer employs a mapping mechanism that converts categorical variables into numerical representations while maintaining a bidirectional mapping for result interpretation.

[2] https://github.com/darpai2o/Transparent-Computing
[3] https://github.com/security0528/PublicArena

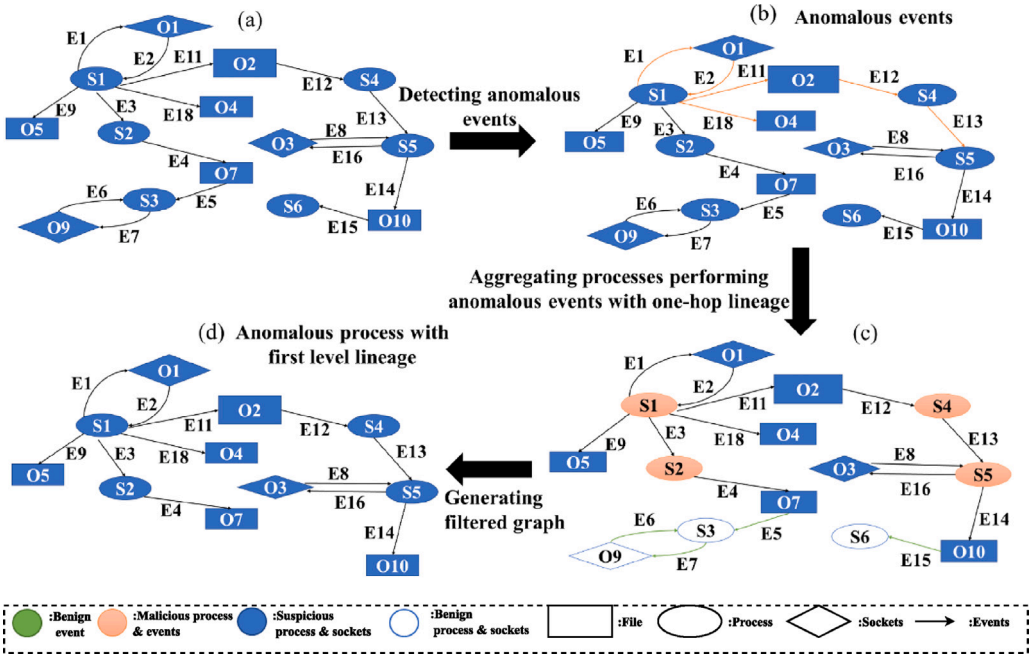


Fig. 2. Deviation analyzer: Detection of event-level anomalies, followed by the addition of the processes and their first-level ancestors and descendants for further analysis.

(b) Outlier detection. The anomalous event detection task involves several key steps. First, we preprocess the system logs by removing duplicate entries to ensure that unique process-object interactions are analyzed. Log records undergo a further preprocessing phase where categorical features are extracted and transformed into numerical representations. Specifically, features such as process identifier, event type, and object identifier $\langle n_i, e_i, d_i \rangle$ are mapped using a frequency-encoder. The features are then standardized using StandardScaler to ensure uniform feature scaling. Anomalies are then identified using the local outlier factor (LOF) algorithm. The core anomaly detection is performed as follows:

$$A(l_i) = \begin{cases} 1 & \text{if } \mathrm{LOF}_k(l_i) > \tau, \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

where τ is the contamination threshold set at 0.1 and k is the number of neighbors set at 20 used for density estimation by the LOF algorithm; these values were set based on an empirical study we performed.

(c) Filtered graph creation. Based on the detected anomalous events (red arrows in Fig. 2b), the deviation analyzer identifies the processes performing these events (red ovals in Fig. 2c). Then, for each anomalous process identified, the deviation analyzer extracts: (1) the process's direct ancestors through fork events, which are identified by matching the object identifier (o_i) with the anomalous process's identifier (p_i), and (2) the process's immediate descendants, found by examining fork events where the anomalous process is the parent, as shown in Fig. 2c (process $S2$). Choosing one-hop lineage stems from a well-known property of system-level provenance graphs where a process is responsible for transferring data between two objects (Inam et al., 2023). This lineage tracking is formalized as:

$$R = \bigcup_i G_i \quad \forall l_i : A(l_i) = 1 \tag{2}$$

where G_i represents the subgraph containing the anomalous process and its one-hop lineage as depicted in Fig. 2d.

Graph Analyzer. The logs are received as anomalous records from the deviation analyzer. We then create a filtered graph by taking each p_i and o_i as nodes and the event as edges between these nodes. Using the filtered graph from the deviation analyzer as input, the graph analyzer examines relationships between processes, identifying potential points of infection, prunes benign nodes, and clusters the suspicious nodes using community detection algorithms (as illustrated in Fig. 3).

(a) Detection of initial infection points. SHIELD focuses on attacks originating from entities through network communication interfaces (sockets), which serve as the connection points between the victim's system and external entities (e.g., a user visiting a malicious website that downloads a payload to the system). As illustrated in Fig. 3, the socket is the object from which the attack was initiated. We refer to all such external sockets as initial infection points. In the example presented in Fig. 3a, socket object *O1* is the initial infection point.

(b) Suspicious tag propagation. The graph analyzer propagates the suspicious tags from the initial infection points to other nodes in the provenance graph; (initially, all of the processes that receive data from the initial infection points are tagged as suspicious.) Then, these tags are propagated to the nodes that receive data from these suspicious processes. Tags are thus propagated to all the entities that exist in the path. A tag propagation path only ends when: (1) a socket object is encountered, or (2) entities other than socket objects do not relay the data they receive. Fig. 3b shows an example of tag propagation paths in the graph.

(c) Pruning non-infected subgraphs. After the tags have been propagated, the graph analyzer prunes all nodes that are not tagged (i.e., nodes that do not contribute to the flow of provenance information, or are completely isolated). This iterative pruning process reduces graph complexity while preserving critical attack-related information. In Fig. 3C, the initial infection originates at socket O1. From this point, the data propagates along several distinct paths: O1→S1→O2→S4→S5→O10, O1→S1→O2→S4→S5→O3, O1→S1→O4, O1→S1 →S2→O7, and O1→S1→O5. As a result of this propagation, all entities involved in these paths are tagged accordingly. Subsequently, file objects O4, O5, O7, and O10 are pruned from the graph, as they represent terminal nodes in the propagation chain and do not contribute to further transmission of the data. Although entity O3 represents the

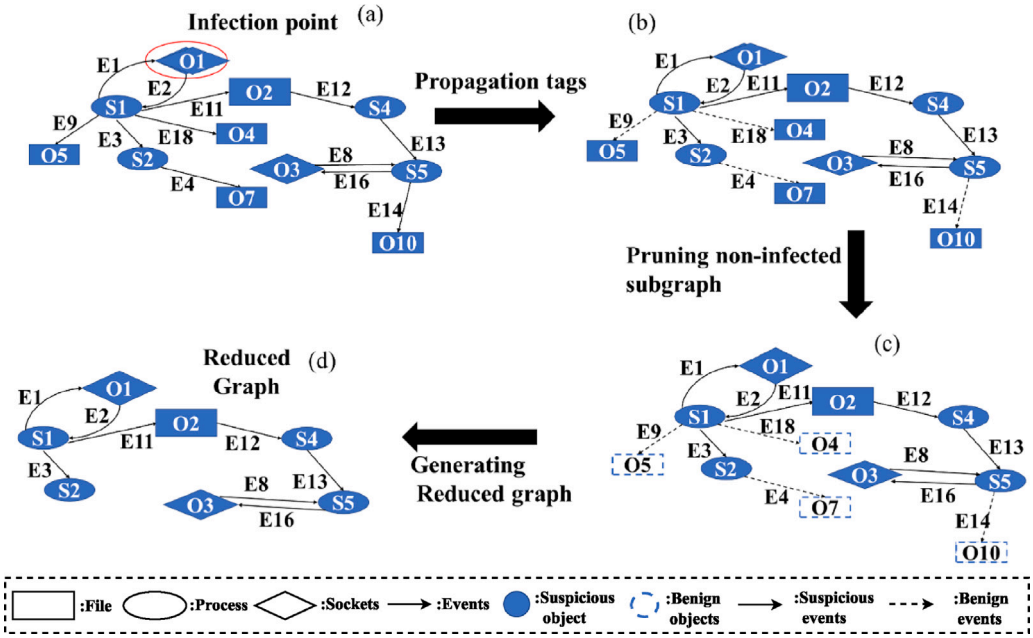


Fig. 3. Graph analyzer: Detection of infection points, followed by tag-propagation and iterative pruning of benign entities, resulting in a reduced graph structure optimized for further analysis.

terminal point of the path O1→S1→O2→S4→S5→O3, it is not pruned from the graph. This is because O3 is an external socket that signifies the culmination of data transfer through multiple system entities and indicates propagation beyond the system's boundaries. In contrast, the other terminal file objects merely denote internal endpoints within the system and do not facilitate further dissemination of the data. Based on this pruning criteria, we can formally define the reduced graph $G_R = (V_R, E_R)$ as:

$$V_R = \{v \in V \mid T(v) = 1\},$$
$$E_R = \{(u, v) \in E \mid u, v \in V_R\}. \tag{3}$$

(d) Community detection. The Louvain algorithm is applied to the reduced graph, clustering related nodes into communities to identify groups of processes that may be working together in coordinated attack activities. We chose this algorithm since: (1) it efficiently handles large, high-dimensional graphs, (2) it is known for producing high-modularity partitions, and (3) prior studies have demonstrated its effectiveness in similar tasks (Pei et al., 2016). This choice is further motivated by a key observation about APT behavior: attack-related activities typically form dense, interconnected communities within the provenance graph, with malicious processes exhibiting higher degrees of interaction among themselves than normal system processes, as demonstrated in Blondel et al. (2008). By leveraging these behavioral patterns, the Louvain algorithm, combined with our temporal correlation engine, effectively identifies these related activities, revealing potential attack sequences that might otherwise remain hidden in the broader system activity.

Continuing with the example from 3, the remaining graph contains entities O_1, S_2, O_2, S_4, $_S5$, O_3, S_2 with their associated edges. The Louvain algorithm partitions this reduced graph into distinct communities based on edge density and modularity. In this case, the algorithm groups these entities into a single community O_1, S_1, O_2, S_4, S_5, O_3, S_2, representing the complete attack behavior within this time window. Each community thus represents a focused, contextually coherent unit of events that can be independently analyzed by the LLM in subsequent stages, enabling more precise reasoning.

LLM Analyzer. The LLM analyzer module constitutes the third module of the SHIELD pipeline. Building on the communities identified by the graph analyzer, the LLM analyzer processes the system events corresponding to the community nodes to detect malicious behaviors. The LLM analyzer examines the relationships and temporal sequences of operations and determines whether an attack has been observed, provides a confidence score, and maps the malicious events to the corresponding stages of the kill chain. When the LLM concludes that the events in the communities are highly suspicious, the corresponding nodes in the reduced graph are tagged with the confidence score; the graph is then traversed and all of the traceable nodes (processes and objects) with their corresponding edges (events) are sent to the temporal correlation engine.

The methodology adopts a three-stage chain-of-thought (CoT) reasoning framework to methodically analyze complex attack scenarios. For each community identified by the graph analyzer, the LLM independently performs the following analysis. In the initial stage, the LLM scans the logs within the community, leveraging its pre-trained knowledge of typical system behavior to identify processes exhibiting atomic deviations from expected patterns. The deviations include, but are not limited to, writes to shared memory segments, the execution of binaries located in non-standard directories, or unexpected outbound network connections. The output of this stage is a candidate set C, C = $\{p_1, \ldots, p_k\}$ that is guaranteed to be exhaustive yet may contain benign outliers. In the second stage, for each candidate process $p \in C$ the LLM analyzes the temporal sequence of actions associated with each flagged process to determine whether the process itself is performing suspicious activities. The decision function is binary: (i) unknown binaries are labeled suspicious by default, and (ii) known binaries must undergo a fine-grained, temporal consistency check in which their current behavior is compared against the LLM's understanding of their expected behavioral patterns. Only processes successfully validated as suspicious constitute the refined malicious set $M \subseteq C$ which substantially reduces the search space for more in-depth analysis in the next stage. The final stage comprises two sub-stages: (a) the first sub-stage is a validation and expansion stage in which the LLM reexamines the events within the community, now filtered to include only elements associated with the malicious set M, and augments this set by incorporating any parent, child, or peer processes contributing to the identified attack chain (for

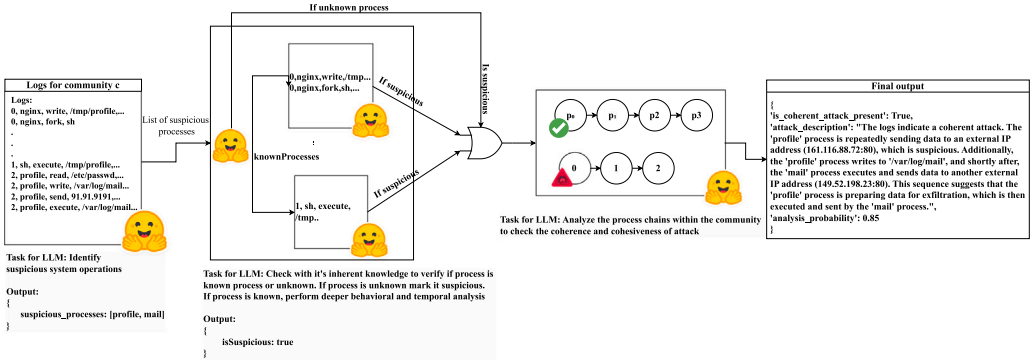


Fig. 4. LLM analyzer: An illustration of the CoT reasoning.

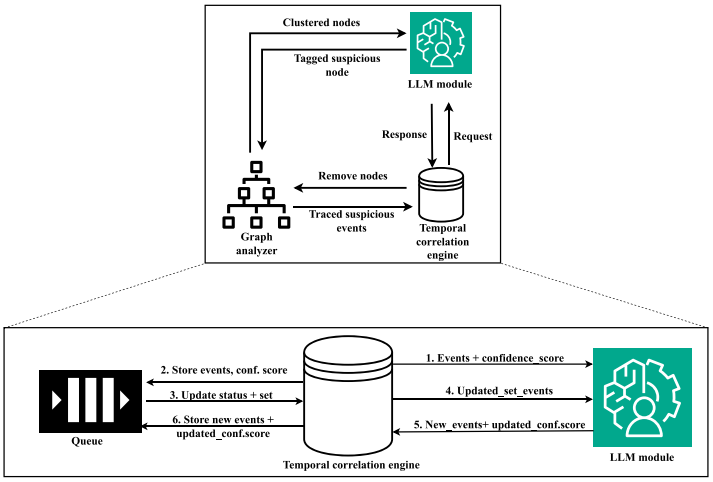


Fig. 5. LLM analyzer and temporal correlation engine: Analyzes system logs, tags malicious nodes, performs graph traversal, adds the tagged nodes to the attack set, and maintains historical context of attack evolution while considering memory constraints.

example, a launcher spawning a downloader); (b) in the second substage, the LLM generates a structured, human-readable incident report. This report systematically presents a calibrated confidence score, a concise attack summary, and set A, consisting of all malicious indicators. This bottom-up progression mirrors the cognitive workflow of expert analysts and is repeated independently for each community produced by the graph analyzer. The reasoning framework has been formally specified in Appendix A; a graphical illustration of the same is provided in Fig. 4.

Following the multi-stage analysis, the LLM assigns confidence scores. Building on prior work showing LLMs perform better with rating scales than categorization (Zhuang et al., 2024), and leveraging Freitas et al.'s use of a 0.9 confidence threshold to forward effective responses (Freitas et al., 2024), we developed a confidence scoring mechanism. Our approach assigns confidence scores (σ_a) based on detected attack sequences: ≥0.9 for complete attack sequences, 0.8–0.9 for partial attack sequences, and 0.7–0.8 for detected suspicious patterns. The optimal alert threshold (δ) for differentiating between primary and secondary queues was determined through a comprehensive sensitivity analysis, detailed further in Appendix D.

SHIELD sets the alert threshold (δ) at 0.8 to focus analyst attention on high-confidence detections of partial or complete attacks, filtering out lower-confidence findings that might lead to alert fatigue. When ($\sigma_a \geq \delta$), the LLM returns a three-part response. First, it generates a comprehensive alert for security analysts, detailing the attack description, suspicious processes, sequence of events, and their mapping to kill chain stages. Second, it tags nodes ($Tag(c)$), marking all suspicious processes identified by the LLM within community c with their respective confidence scores.

Finally, by traversing the graph ($Trace(c)$), it examines the reduced graph G_r from the graph analyzer module, propagating confidence scores to all nodes reachable from the tagged suspicious processes. If ($\sigma_a < \delta$), the system limits its response to tagging suspicious nodes and traversing the graph. The response can be formalized as:

$$Response(c, \sigma_a) = \begin{cases} GenerateAlert(c) + Tag(c) + Trace(c) & \text{if } \sigma_a \geq \delta, \\ Tag(c) + Trace(c) & \text{otherwise.} \end{cases} \tag{4}$$

Tagged and traced nodes are collected to form an attack event set T and forwarded to the correlation engine (see Fig. 5).

Temporal Correlation Engine. The temporal correlation engine serves as SHIELD's orchestration module, managing both the prioritization of detected threats and the temporal aspects of long-term attack monitoring. It processes the attack set received from the LLM analyzer through two main mechanisms: dynamic set integration for threat assessment and rolling provenance updates for memory management.

Table 1
Provenance statistics of the datasets used in our evaluation.

Dataset	Train Split Time	# Attacks	# Nodes	# Edges	# Attack Nodes	# Attack Edges
DARPA-E3-CADETS	2018-04-06 11:00	3	178,965	10.1	68	1248
DARPA-E3-THEIA	2018-04-09 21:25	1	690,105	32.4 M	118	3119
DARPA-E3-CLEARSCOPE	2018-04-09 23:55	1	68,549	9.7	41	647
OpTC	2019-09-20 23:55	1	9,485,265	75 M	46	7 K
Public Arena	2022-05-12 23:55	1	25,533	6 M	6	3548
Blind Eagle	2024-11-06 16:18:00	2	688	6149	37	263

(a) Dynamic set integration and risk stratification. This process serves as the primary threat assessment mechanism by consolidating analysis results into a global attack set that tracks both ongoing and historical attacks. During this stage, behavioral correlation analysis leverages the confidence scores (σ_a) assigned by the LLM analyzer to identify related attack patterns, merging and filtering sets to consolidate partial APT activities into comprehensive attack chains, sorted by timestamps. After merging, SHIELD employs two core adjustment mechanisms that modify the initial LLM-assigned confidence scores: (1) Decay, which is applied when a tagged set continues to exhibit benign behavior over subsequent analysis windows. Without further suspicious activities supporting the initial classification, the confidence score gradually decreases. Once σ_a falls below 0.7, the set is removed from active analysis and its nodes become eligible for pruning. (2) Boosting mechanism, which is activated when a set initially receives a lower confidence score ($0.7 \leq \sigma_a < 0.8$) due to early-stage attack indicators. As additional stages of the attack manifest in subsequent windows, the score increases proportionally, reflecting mounting evidence of malicious intent. This approach effectively identifies APTs that evolve slowly over time.

During set integration, SHIELD maintains the same attack set structure T and the confidence score threshold σ_a (see LLM analyzer). The engine's queue management directly aligns with these thresholds: sets with $\sigma_a \geq 0.8$ enter the primary queue for immediate analyst attention, while those with $0.7 \leq \sigma_a < 0.8$ remain in the secondary queue for continued observation. SHIELD continuously applies decay or boosting during subsequent integrations, adjusting confidence scores based on emerging evidence. This dual-queue prioritization process enables efficient allocation of analyst resources toward high-confidence attack chains while maintaining visibility of potentially developing threats.

(b) Rolling provenance graph updates. SHIELD implements a rolling provenance graph to manage memory usage when tracking long-term attacks. The graph continuously incorporates new process and event nodes from the system logs, with suspicious nodes being tagged with confidence scores by the LLM analyzer. For memory efficiency, SHIELD removes untagged nodes after analyzing the time window r and removes tagged nodes at regular intervals. This pruning is possible, because once suspicious activities are detected, their complete behavioral patterns and relationships are preserved in the attack sets T, making the actual graph nodes redundant after analysis. Through this efficient memory management technique, SHIELD maintains comprehensive attack tracking capabilities for APTs while preventing unbounded graph growth. In Section 6, we present an example demonstrating how SHIELD successfully identifies and tracks sophisticated attacks spanning multiple days.

4. Evaluation setup

Implementation. We implemented SHIELD in Python 3.11, using custom log parsers for data extraction. The deviation analyzer employs scikit-learn's LOF model, and the graph analyzer uses NetworkX's (Hagberg et al., 2008) MultiDiGraph with the Louvain algorithm.[4] We developed a custom filtering mechanism to filter suspicious file operations, network communications, and process-file operations. For the LLM, we deployed Qwen 2.5 (32B) in 8-bit quantized mode on an NVIDIA RTX 6000 GPU. For the sake of reproducability, we have provided the code in GitHub repository.[5]

Datasets Our evaluation leverages three system-level audit log datasets (THEIA, CADETS, Public Arena) and a high-level application log dataset (BlindEagle) in diverse environments (in Table 1 we provide information on the datasets):

- *DARPA eng.3 THEIA:* A comprehensive Ubuntu Linux dataset comprising 106 million system audit logs (40 GB) collected over 12 days. It contains two stages of a successful APT attack instance and one failed attempt, with the successful attack spanning three days—(ideal for evaluating long-duration, multi-stage APT detection capabilities).
- *DARPA eng.3 CADETS:* A FreeBSD-based dataset containing 42 million audit logs (25 GB) captured over 12 days, featuring four APT attack instances (three successful and one failed). This dataset complements THEIA by providing attack patterns in a different Unix-like environment.
- *DARPA eng.3 CLEARSCOPE:* A android based dataset containing 11 million audit logs spanning across 12 days featuring one partially successful attack and one failed attempt.
- *DARPA OpTC:* A windows based dataset containing 100 million records, collected over various machine in a span of 10 days. It complements the other DARPA dataset.
- *Public Arena:* A Windows-focused dataset collected from two hosts in a simulated public cloud environment over six days. It contains one instance of an APT attack. With 16 million audit records (7 GB), it enables cross-platform validation of our framework.
- *High-Level Application Log:* To validate SHIELD's applicability in enterprise environments, we developed an in-house testbed emulating the Blind Eagle APT campaign. The experiment ran for 60 min, generating 6100 Splunk logs across three Windows workstations running Windows 10 and Windows 11. We simulated the attack by compromising two out of the workstations while incorporating realistic background user activities, such as web browsing, document editing, and email usage, in our controlled environment. This setup enabled us to evaluate SHIELD's efficacy in detecting APT patterns and distinguishing them from benign user behaviors.

To further ensure that our evaluation reflects real-world scenarios and to minimize data leakage, we split each dataset into training and testing windows. For the CADETS dataset, the very first malicious events occur on April 6, but are followed by a three-day span of benign activity (April 7–9). Several baselines (Jiang et al., 2025; Wang et al., 2022) have used this post-attack window for self-tuning their encoders effectively peeking into the future: their encoders are updated with events that occur after the initial compromise and may still carry residual attack artifacts. As a result, reported precision and recall numbers can be artificially high, since the model has already seen malicious

[4] https://github.com/taynaud/python-louvain

[5] https://github.com/SHIELD-EAAI/SHIELD

behavior during tuning. In SHIELD, we enforce a strict chronological cutoff at $t_{\text{first_attack}}$, tuning our LOF detector only on the 30% of events before April 6 and then running entirely online thereafter. We apply the same procedure to every dataset reserving the $t < t_{\text{first_attack}}$ logs for tuning and subsequent logs are part of the testing.

DARPA released the ground truths for CADETS, THEIA, CLEARSCOPE and OpTC datasets containing IoCs and timestamps but not specific attack events. We have utilized this ground truth for evaluating and comparing SHIELD against other methods for time-window based evaluation. However to evaluate and compare SHIELD on coarse grained entity-level detection we utilized the ground truth that has been marked by SOTA methods including KAIROS (Cheng et al., 2023) and ORTHRUS (Jiang et al., 2025) For the Public Arena and Blind Eagle datasets we already had the ground truths with the identified attack events. Table 1 summarizes the characteristics of the datasets, including key provenance statistics (e.g., number of edges, nodes, malicious entities), and overall dataset size. These datasets and their ground truth mappings served as the foundation for our comprehensive evaluation methodology.

Evaluation Metrics. To evaluate SHIELD's performance, we employed a two-phase analysis process: coarse-grained (time-window level) detection and fine-grained (event-level) detection. This dual-layer approach enables comprehensive evaluation of the system's detection capabilities. We used standard performance metrics: precision, recall, and the F1-score. At the event level, we focused on measuring the alert quality and assess SHIELD's effectiveness in reducing alert fatigue, a critical consideration for practical deployment in security operations. We compare our framework's performance to that of state-of-the-art APT detection systems, including UNICORN (Han et al., 2020), DeepLog (Du et al., 2017), and KAIROS (Cheng et al., 2023), ORTHRUS (Jiang et al., 2025) and FLASH (Rehman et al., 2024) For fairness, we used the original implementations and hyperparameters provided by the respective authors.

5. Evaluation results

Performance Analysis of Deviation Analyzer. We start by evaluating the performance of the deviation analyzer, focusing on its detection capabilities and log reduction efficiency. The deviation analyzer demonstrates exceptional detection capabilities with 100% recall on all datasets, successfully identifying every known attack event while maintaining a minimum log reduction of approximately 24 times across all datasets. Its adaptive thresholding mechanism (i.e., the contamination threshold) effectively distinguishes malicious patterns from benign system behaviors. The analyzer's efficiency in processing large-scale log data with minimal computational overhead makes it particularly valuable for real-time APT detection in enterprise environments.

However, detection alone is insufficient for operational security workflows. As detailed in Appendix B, our comparative study of events flagged by SHIELD versus LOF reveals a critical limitation: while statistical models achieve strong baseline detection performance, they generate high false positives. These noisy outputs overwhelm analysts during post-detection investigation, rendering such models impractical for triage or root-cause analysis. SHIELD addresses this gap by coupling detection with context-aware graph based filtering and LLM aided analysis, ensuring not just identification of threats but actionable intelligence for responders.

Performance Analysis of Graph Analyzer. Next, we examine the graph analyzer's effectiveness in preserving attack events while significantly reducing the number of logs on diverse datasets. The graph analyzer demonstrates effective capabilities in preserving attack events while achieving significant log reduction on all datasets. We achieved a mean reduction of 95.58% across the datasets, with standard deviations ranging from 1.98% to 6.59%, substantially reducing the computational overhead in analysis. In terms of recall, the analyzer maintains near-perfect attack event retention across datasets with rates consistently exceeding 99%. This exceptional performance is demonstrated by minimal attack event loss – only three events across millions of log entries – showcasing the module's ability to preserve critical event information even in complex environments with diverse log patterns. These results suggest significant practical implications for large-scale security monitoring systems, where it can substantially reduce storage and processing requirements while maintaining comprehensive threat detection capabilities.

Time Window-Based Performance Analysis of SHIELD. Having established the effectiveness of two of the modules, we evaluate SHIELD's overall performance using a systematic time window-based approach. We employed a sliding window mechanism with 30-minute intervals and a 15-minute step size. This allowed for continuous monitoring and evaluation of system behavior. The CADETS dataset was divided into 714 windows, of which only 12 contained attack sequences, illustrating the imbalanced nature of the data. Similarly, the THEIA dataset comprised 303 windows, with nine containing attacks, reflecting a similar skew. Clearscope had three and OpTC had only two windows that contained the attack events.

In this initial evaluation phase, we considered GPT-4o as a potential LLM. However, using GPT-4o presents inherent challenges for threat detection scenarios. As a closed-source model hosted by a third party, transmitting sensitive log data to external servers raises significant privacy and security concerns. Furthermore, GPT-4o operates on a subscription-based pricing model, making it cost-prohibitive for continuous, large-scale log analysis tasks. Due to these limitations, we focused our assessment on several open-source local language models that could be deployed on-premises. Among the candidates—Qwen 2.5 (32B), EXAONE 3.5 (32B), and LLaMA 3.1 (8B). Qwen 2.5 demonstrated superior and consistent performance across all datasets. Detailed results of our comparative study for model selection are provided in Appendix C.

Furthermore, we compare SHIELD's performance for time-window based detection against state-of-the-art detection systems, including KAIROS, UNICORN, DeepLog and LogGPT on the CADETS, THEIA, CLEARSCOPE and OpTC datasets by adapting them to our time-window based approach. While Public Arena's parsers were unavailable and Blind Eagle's high-level log format was incompatible with other provenance-based Intrusion Detection System, the comprehensive comparison on compatible datasets demonstrates SHIELD's competitive performance on multiple key metrics, as summarized in Table 2.

Entity-Level Performance Analysis of SHIELD. While the window-based metrics provide a high-level view of SHIELD's detection capabilities, a more granular event-based analysis is necessary to evaluate its practical utility for security operations. SHIELD's primary objective is to alleviate the workload of security analysts. Instead of requiring analysts to perform extensive triage on large volumes of false positive alerts, SHIELD automatically summarizes what it deems to be an attack, enabling analysts to quickly evaluate whether malicious activity is present in the system. Even in scenarios where alerts might be false positives, the consolidated summary containing IoCs can help analysts decide if further action is necessary. Since SHIELD bases its summaries on identified attack events, it is critical to examine these events to understand the extent and accuracy of SHIELD's detection capabilities.

On all datasets we evaluated, SHIELD successfully identified all attacks while maintaining balanced precision and recall metrics. SHIELD demonstrated superior performance, achieving the highest F1 Score among all evaluated approaches on all datasets.

To further benchmark SHIELD, we compared its entity-level detection capabilities against other methods. Table 3 presents a detailed comparison of SHIELD with other SOTA like ORTHRUS, Flash, KAIROS, Unicorn etc. The results demonstrate SHIELD's superior recall, maintaining balanced precision across all datasets. The ground truth for our

Table 2
Comparison of time-window based detection performance of different methods on CADETS, THEIA, CLEARSCOPE and OpTC datasets.

Method	Dataset	Attack Identified	Confusion Matrix				Performance Metrics		
			TP	FN	FP	TN	Precision	Recall	F1-Score
SHIELD	CADETS	3/3	7	5	17	685	0.29	0.58	0.39
	THEIA	2/2	6	3	4	290	0.60	0.67	0.63
	CLEARSCOPE	1/1	2	1	12	271	0.14	0.67	0.24
	OpTC	1/1	2	0	2	18	0.50	1.00	0.67
KAIROS	CADETS	2/3	9	3	10	692	0.64	0.75	0.69
	THEIA	2/2	5	4	11	283	0.31	0.55	0.40
	CLEARSCOPE	0/1	0	3	14	269	0.07	0.33	0.11
	OpTC	1/1	1	1	4	16	0.20	0.50	0.29
Unicorn	CADETS	3/3	4	8	28	674	0.13	0.33	0.18
	THEIA	2/2	4	5	23	271	0.15	0.44	0.22
	CLEARSCOPE	1/1	2	1	68	215	0.03	0.67	0.05
	OpTC	1/1	2	0	4	19	0.67	1.00	0.80
DeepLog	CADETS	3/3	9	3	683	19	0.01	0.75	0.03
	THEIA	2/2	4	5	60	234	0.06	0.44	0.11
	CLEARSCOPE	1/1	1	2	81	202	0.01	0.33	0.02
	OpTC	1/1	1	1	8	12	0.11	0.50	0.18
LogGPT	CADETS	3/3	12	0	689	13	0.02	1.00	0.03
	THEIA	2/2	9	0	287	7	0.03	1.00	0.06
	CLEARSCOPE	1/1	3	0	263	20	0.01	1.00	0.02
	OpTC	1/1	2	0	19	1	0.10	1.00	0.17

comparison has been sourced from ORTHRUS (Jiang et al., 2025) to remain fair with all methods.

Attack Summary Quality Analysis. SHIELD's ability to generate interpretable and structured attack summaries is one of its greatest advantages, complementing its quantitative detection efficacy. Below, we present the attack summaries and analyses for the THEIA and BlindEagle datasets, demonstrating SHIELD's alignment with the cyber kill chain framework and its provision of detailed, contextual insights.

Attack Summary for THEIA Dataset

The sequence of events indicates a coherent attack involving multiple processes interacting with user-level objects and external IP addresses. The attack appears to follow these stages in the cyber kill chain:

- **Reconnaissance:** `fluxbox` reads various Firefoxrelated files (cookies, certificates, crash reports), gathering system and user information.
- **Delivery:** `fluxbox` communicates with external IPs (61.130.69.232, 141.43.176.203), suggesting C2 traffic.
- **Exploitation:** `firefox` writes to and executes `/home-/admin/clean` and `/home/admin/profile`, which are then run by `clean` and `profile`, respectively, also contacting 146.153.68.151 and 161.116.88.72.
- **Installation:** `firefox` writes `/etc/firefox/native-messaging-hosts/gtcache`, later executed by `sh`, indicating a persistent backdoor.
- **Command and Control:** `gtcache` communicates with 146.153.68.151 and writes to `/dev/glx_alsa_675`.
- **Actions on Objectives:** `profile` writes to `/var/log/mail`, then `mail` executes and contacts 149.52.198.23, indicating potential exfiltration.

THEIA Dataset Summary Analysis On the THEIA dataset, SHIELD mapped the entire attack progression. For the delivery phase SHIELD accurately identified shellcode servers (61.130.69.232, 141.43.176.203), It then accurately traced the exploitation chain through the `firefox`, `clean`, and `profile` processes. The analysis captured the installation of persistence capturing persistence via `/etc/firefox/native-messaging-hosts/ gtcache`, and effectively tracked C2 activities. Additionally, SHIELD identified potential data exfiltration patterns through the `profile` and `mail` processes, demonstrating its ability to thoroughly trace the attack's impact.

Attack Summary for BlindEagle Dataset

The attack sequence again follows the kill chain:

- **Reconnaissance**: The `explorer.exe` process reads from the user's Downloads directory, indicating potential enumeration of files.
- **Delivery**: The `cmd.exe` process, spawned by `AutoHotkey64.exe`, executes a script (`downloadfac-with-curl.bat`) and interacts with external entities to download malicious payloads.
- **Exploitation**: The `wscript.exe` process, triggered by `AutoHotkey64.exe`, executes a VBScript (`factura-228447578537.pdf.vbs`) that includes encoded commands to download and execute additional payloads using `cmd.exe` and `powershell.exe`.
- **Installation**: Various processes such as `power-shell.exe` load potentially malicious assemblies and attempt to establish persistence by interacting with system-level objects.
- **Command and Control**: The downloaded payloads and commands suggest ongoing communication with a remote command-and-control server (CDN FIBER).
- **Actions on Objectives**: Processes like `Auto-Hotkey64.exe` and `explorer.exe` further interact with files and directories, indicating potential data exfiltration or lateral movement.

BlindEagle Dataset Summary Analysis In BlindEagle, SHIELD pinpointed delivery via `download-fac-with-curl.bat`, detailed VBScript exploitation (`factura-228447578537.pdf.vbs`), and traced C2 to the CDN FIBER server, demonstrating comprehensive stage coverage.

Table 3
Comparison of entity-level detection performance of different methods on CADETS, THEIA, CLEARSCOPE and OpTC datasets.

Dataset	System	Confusion Matrix				Performance Metrics		
		TP	FP	TN	FN	Precision	Recall	F1-Score
CADETS	SHIELD	35	0	268 k	33	**1.000**	0.515	**0.680**
	ORTHRUS-full	25	23	268 k	43	0.521	0.368	0.431
	ORTHRUS-ano	33	0	268 k	149	**1.000**	0.18	0.31
	Kairos	15	9	268 k	53	0.625	0.221	0.326
	Threatrace	61	252 k	16 k	7	0.000	0.897	0.000
	MAGIC	63	80 k	188 k	5	0.001	**0.926**	0.002
	Flash	13	2.4 k	266 k	55	0.005	0.191	0.010
THEIA	SHIELD	76	19	699 k	42	0.800	0.644	**0.713**
	ORTHRUS-full	48	11	699 k	70	0.814	0.407	0.541
	ORTHRUS-ano	51	10	699 k	106	**0.84**	0.32	0.47
	Kairos	37	78	699 k	81	0.322	0.314	0.318
	Threatrace	88	672 k	27 k	30	0.000	0.746	0.000
	MAGIC	115	395 k	304 k	3	0.000	**0.975**	0.001
	Flash	22	32 k	667 k	96	0.001	0.186	0.001
CLEARSCOPE	SHIELD	2	1	111 k	39	**0.667**	0.049	**0.091**
	ORTHRUS-full	2	6	111 k	39	0.250	0.049	0.082
	ORTHRUS-ano	2	2	111 k	56	0.50	0.03	0.06
	Kairos	0	7	111 k	41	0.000	0.000	0.000
	Threatrace	41	88 k	24 k	0	0.000	**1.000**	0.001
	MAGIC	40	102 k	9.6 k	1	0.000	0.976	0.001
	Flash	0	15 k	96 k	41	0.000	0.000	0.000
OpTc	SHIELD	46	19	7 k	0	0.708	**1.000**	**0.829**
	ORTHRUS-full	36	21	7 k	10	0.632	0.783	0.699
	ORTHRUS-ano	34	4	7 k	43	**0.89**	0.44	0.59
	Kairos	12	127	7 k	34	0.086	0.261	0.130
	Threatrace	46	6 k	732	0	0.007	**1.000**	0.014
	MAGIC	46	4 k	3 k	0	0.011	**1.000**	0.022
	Flash	10	444	7 k	36	0.022	0.217	0.040

6. Discussion

Label Encoder vs Frequency Encoder. SHIELD is designed as a modular system, enabling experimentation with multiple encoding strategies. In addition to a label encoder, we evaluated a frequency-based encoder. While the label encoder is computationally more efficient and less memory intensive, it has already yielded satisfactory performance, making it initially attractive from a runtime perspective. The frequency encoder, on the other hand, is better suited for capturing the rarity of objects, reflecting that anomalous elements are seldom accessed (Li et al., 2024; Hassan et al., 2019b). This property allows the frequency encoder to provide richer contextual information than the label encoder, which is consistent with our experimental findings. When integrated into the pipeline, the frequency encoder achieved improved runtime performance and enhanced log reduction in the deviation analyzer from approximately 20× to 24×, at the cost of increased computational overhead.

Attack Detection Effectiveness. For instance, on the THEIA dataset, the TCE tracks a single attack across three days. On the first day it observes *firefox* exploiting a process called *profile*, which then emits a C2 beacon to download the second payload; the LLM assigns a confidence score σ_a of 0.85 to these set of events that it analyzes and a new set of these events is created and placed in the primary queue. No additional malicious activity is detected for this chain, so the merged set is resubmitted unchanged, and the LLM returns $\sigma_a = 0.85$, keeping the set in the primary queue. However, on the third day, the TCE encounters new events in which *profile* downloads a malicious payload into */var/log/mail* and forks a child process called *mail* and correlates these events with the earlier set of events. After merging these events the complete graph is resubmitted to the LLM, which now raises the score to $\sigma_a = 0.91$, so the fully confirmed attack remains in the primary queue. For the CADETS dataset, the TCE follows the same procedure. On the first day it encounters a chain consisting of *sshd*, *wget*, and *alpine* exhibiting a suspicious pattern wherein *sshd* executes *bash*, which in turn executes *alpine* and *wget* and writes to *.bash_history*. The *alpine* process accesses sensitive files like */var/mail/user* and *wget* performs external network communications. The LLM assigns σ_a of 0.85 and the set enters the primary queue. and the set enters the primary queue. A few hours later, additional processes performing similar activities are also added to the set, so when the merged set is resubmitted, the LLM lowers the score to $\sigma_a = 0.77$, prompting the engine to move the set to the secondary queue. Benign activity continues on the next day, and they are also assigned by the LLM to the same set, leading the LLM to reduce the score further to $\sigma_a = 0.49$, which falls below the 0.7 threshold, so the engine removes the set from active tracking and marks its nodes for pruning. This decay mechanism effectively addresses alert fatigue in security operations by ensuring that analysts focus on high-confidence threats.

While other methods like KAIROS calculates the reconstruction error, based on the presence and frequency of anomalous edges within a given time window. When an attack generates a high volume of anomalous nodes and edges, KAIROS effectively captures it with high precision. However, in scenarios where the anomalous activity is sparse, KAIROS tends to miss such events, resulting in detection failures. In contrast, SHIELD remains robust in these situations, as it does not rely on anomaly frequency. Instead, it leverages causal relationships between events and performs temporal correlation analysis, enabling it to detect attacks even when their correlation is weak or spread across time. This is reflected in the results presented in Table 2; as can be seen, SHIELD detects all of the attacks in the CADETS dataset whereas KAIROS misses one attack completely.

Hallucination Mitigation. A key challenge when utilizing LLMs is the risk of hallucination. SHIELD's multi-stage architecture was explicitly designed to mitigate this risk through multiple complementary mechanisms.

First, Deviation Analyzer filters noise from raw logs, ensuring the LLM only analyzes statistically anomalous events rather than arbitrary system activity. Second, Graph Analyzer provides explicit causal relationships (process→event→file) rather than implicit correlations, grounding the LLM's reasoning. Third, the LLM Analyzer employs CoT prompting that structures reasoning into progressive validation stages,

requiring the model to justify each step rather than making holistic judgments that are more prone to hallucination. Finally, the TCE acts as a post-hoc hallucination filter: if the LLM incorrectly flags benign behavior, confidence scores decay over subsequent time windows as no corroborating evidence emerges. This decay mechanism effectively self-corrects false positives that may arise from LLM hallucinations. The effectiveness of this design is reflected in our results as SHIELD achieves high precision on all datasets.

Zero-Day Attack Detection Capabilities. Our evaluation on four datasets, including real-world and benchmark scenarios, demonstrated SHIELD's effectiveness in detecting previously unknown attacks. Since attackers must progress through certain essential stages regardless of their specific techniques, SHIELD can identify suspicious patterns at the kernel level even when confronted with novel attack methods. SHIELD's success in zero-day detection relies on two key factors: the invariant nature of kernel-level attack traces and the LLM's comprehensive understanding of system behavior patterns. While attack implementations may vary, the underlying progression through attack stages creates detectable patterns that SHIELD can recognize without prior exposure to specific variants. Through these capabilities, SHIELD demonstrates a significant advancement in automated attack detection and investigation. The framework's ability to correlate events across extended time periods, manage false positives, and detect novel attack patterns makes it as a valuable tool for modern security operations.

Failure Analysis. To better understand SHIELD's performance, we examined its performance with respect to FPs, FNs, and edge cases across the evaluated datasets. On the CADETS dataset, FPs primarily stem from benign activities that structurally resemble attack patterns. The FPs are further compounded by the sliding window mechanism. For instance, a sequence where *sshd* executes *bash*, which in turn runs *alpine* and *wget* while writing to *.bash_history*, can trigger high initial attack confidence scores ($\delta = 0.85$). Here, *alpine* accesses sensitive files (e.g., */var/mail/user*), and *wget* engages in external network communications, mimicking known attack behaviors. The TCE mitigates this by mapping repeated benign activities to a single attack set and decaying scores over time (e.g., from 0.85 to 0.75). Once below the alert threshold (0.8), these events shift to a secondary monitoring queue, reducing alert fatigue. However, overlapping windows (e.g., 08:30–09:00 and 08:45–09:15) exacerbate FPs, as shared events (08:45–09:00) are flagged redundantly, inflating FP rate. Similar FP patterns arise on the THEIA and Public Arena datasets due to noisy background activities, although recall remains perfect (1.00) with all attacks detected. Clustering and LLM-based reasoning effectively prune these FPs. FNs occur in partial attack overlaps, such as when an attack spans windows but only exhibits subtle patterns (e.g., initial reconnaissance without full exploitation), These are retained in the secondary queue for further monitoring, where TCE groups related events and boosts the confidence score when further evidence is encountered by it. Once the confidence score crosses the alert threshold, the alert is sent to the analyst for review. Edge cases, like benign anomalies, are typically pruned during graph analysis if they lack propagation paths from infection points or their confidence score gets decayed eventually when no additional evidence is encountered, if initially flagged by mistake.

Real-World Resilience and Integration. To evaluate SHIELD's robustness against log degradation scenarios commonly encountered in production environments, we conducted two tests simulating different levels of log loss and corruption.

In the first test, we simulated moderate degradation by removing 30% of non-key events, mimicking typical log failures. To simulate this we randomly removed 11 of the 37 non-critical events while preserving all 20 key attack phase indicators. The LLM was able to detect the attack, and based on the summary generated, we inferred that detection performance remained unchanged, as the preserved key phases enabled the LLM to map the complete kill chain successfully. The TCE framework maintained event-level consistency throughout the analysis, demonstrating strong resilience to non-critical log loss, a common occurrence in intermittent logging environments.

The second test evaluated a moderately critical degradation scenario where key attack events are lost due to corruption. To simulate this we removed 30% of the key attack phases. The LLM was still able to detect the attack, and based on the summary, we inferred that the LLM successfully reconstructed the attack pattern from the surviving key fragments, such as the execution of the XIM process, from surviving data fragments of file write and file modify patterns. This capability allowed SHIELD to accurately identify attack sequences even when key execution records were unavailable. Additionally, the TCE improved the results by merging related attack events across multiple time windows in a single attack set. These findings highlight SHIELD's ability to maintain high detection accuracy under conditions of moderate loss of key attack events.

SHIELD facilitates easy integration with SOC, where it generates high-confidence alerts with interpretable summaries mapped to the APT kill chain, which can reduce alert fatigue; the mapped summaries can further enhance the efficiency of SOC teams. For example, alerts could be exported to SIEM systems (e.g., Splunk or ELK Stack) via API calls or log forwarding agents for correlation with other security events, using SHIELD's JSON outputs as custom inputs to Splunk's HTTP Event Collector or ELK's Filebeat. For SOAR platforms, integration can occur through webhooks that trigger playbooks based on alert scores and kill-chain stages, automating responses like endpoint isolation while allowing analysts to review LLM-generated summaries. This can be done via API endpoints or webhooks for real-time forwarding, with SHIELD's modular design enabling containerized deployment for scalable integration in cloud-based SOCs.

Limitations. SHIELD employs a 15-minute sliding window for attack detection, introducing a maximum detection delay of 15 min in edge cases. However, this delay remains well within acceptable operational parameters, as it significantly outperforms typical SOC mean time to resolve metrics in real-world deployment.[6] From an architectural standpoint, SHIELD's multi-stage pipeline, while crucial for comprehensive threat detection, does present substantial computational demands. Specifically, the local deployment of our chosen LLM necessitates significant GPU resources. The LLM Analyzer and Temporal Correlation Engine are the two most computationally intensive components. These components are only invoked upon anomaly detection, at which time they are invoked asynchronously; they can, however, be offloaded to a centralized server. In terms of computation, a GPU like NVIDIA RTX 6000 with 48 GB VRAM, capable of running our 32B model in 8-bit mode, costs approximately $7000 an amount that would likely be just a small portion of a SOC's overall budget and feasible for deployment.[7] This computational power is rapidly becoming a "commodity" in SOC environments where it is used to facilitate advanced machine learning detection capabilities. We also note that our selection of open, locally deployed models is mainly to avoid the need to send sensitive organizational logs to third-party, hosted models.

7. Conclusions and future work

In this paper, we presented SHIELD, an innovative framework that combines anomaly detection and graph analysis with LLM reasoning for attack detection, investigation, and contextual analysis. Future work can explore integrating automated incident response capabilities using contextual insights and addressing the challenges of parsing high-level logs, as demonstrated in our Blind Eagle proof-of-concept. While our approach effectively analyzes high-level logs, its success depends on complete log capture, highlighting the need to overcome challenges like logging mis-configurations or incomplete capture.

[6] https://www.ibm.com/reports/data-breach

[7] https://www.lumificyber.com/blog/what-does-it-cost-to-build-a-security-operations-center-soc/

CRediT authorship contribution statement

Parth Atulbhai Gandhi: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Investigation, Data curation, Conceptualization. **Prasanna N. Wudali:** Writing – review & editing, Software, Data curation. **Yonatan Amaru:** Writing – review & editing, Validation, Software, Methodology, Data curation. **Akansha Shukla:** Writing – review & editing, Visualization, Software, Data curation. **Yuval Elovici:** Writing – review & editing, Supervision, Resources, Conceptualization. **Asaf Shabtai:** Writing – review & editing, Validation, Supervision, Resources, Methodology, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Asaf Shabtai has patent pending to BGN. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Chain-of-thought detection algorithm

Algorithm 1 Chain-of-Thought detection

Input: Communities C, LLM model M
Output: Attack alerts, summaries, tagged processes

for each community $c \in C$ **do**
  $logs \leftarrow$ GetLogsFromNodes(c)
  $suspiciousProcesses \leftarrow$ CheckKnownBehavior($logs, M$)
  if $|suspiciousProcesses|$ **then**
    $deviationFound \leftarrow$ **False**
    $temporalPatterns \leftarrow \emptyset$
    for each process $p \in suspiciousProcesses$ **do**
      $processLogs \leftarrow$ GetProcessLogs(p)
      $deviation \leftarrow$ AnalyzeBehavior($processLogs, M$)
      if $deviation \neq \emptyset$ **then**
        $deviationFound \leftarrow$ **True**
        $temporalPatterns[p] \leftarrow deviation$
      end if
    end for
    if $deviationFound$ **then**
      $attackChain \leftarrow$ ExtractProcessChain($suspiciousProcesses, temporalPatterns, logs$)
      if $attackChain \neq \emptyset$ **then**
        $(score, summary, tagged) \leftarrow$ AnalyzeChain($attackChain, M$)
        if $score \geq 0.8$ **then**
          return RaiseAlert(), $summary$, $tagged$, $score$
        else if $0.7 \leq score < 0.8$ **then**
          return $tagged$, $score$
        end if
      end if
    end if
  end if
end for
return $\emptyset$

Appendix B. Comparative study of event generation between LOF vs. SHIELD

See Table 4.

Appendix C. Comparative study of LLM models

See Table 5.

Table 4
Events Flagged by LOF vs. SHIELD Across Attack Scenarios.

Dataset	Attack	LOF Events	SHIELD Events
CADETS	Attack 1	3541	13
	Attack 2	5357	25
	Attack 3	3945	19
THEIA	Attack 1	6086	79
PublicArena	Attack 1	6696	274
BlindEagle	Attack 1	728	66

Appendix D. Sensitivity analysis of threshold

See Table 6.

Appendix E. Component analysis and ablation experiments

We conducted systematic component analysis to evaluate the impact of different modules on SHIELD's overall performance. These experiments were performed on components that can be meaningfully isolated without fundamentally altering the system's operational logic. All ablation experiments were conducted on the CADETS dataset, evaluating precision, recall, and runtime efficiency.

E.1. Deviation analyzer impact

Isolating the Deviation Analyzer significantly impacts both performance and efficiency. Runtime increases by 4–6 times (from approximately 55 min to 232 min) as unfiltered raw logs place substantial computational burden on graph analyzer. Precision drops by 13% from 1.0 to 0.87 on the CADETS dataset. This demonstrates the Deviation Analyzer's critical role in computational efficiency and noise reduction by filtering irrelevant system events before they propagate through the analysis pipeline.

E.2. Temporal correlation engine impact

Isolating the Temporal Correlation Engine eliminates SHIELD's ability to track attacks across time windows, reducing performance to standalone time-windowed analysis levels. Precision drops dramatically to 0.29, and recall decreases to 0.58, effectively rendering the system unable to maintain attack context across temporal boundaries. Without boosting and decay mechanisms, analysts face increased workload from investigating unmerged partial attacks and false positives that would normally be correlated or filtered over time.

E.3. Architectural constraints

The Graph Analyzer and LLM Analyzer represent a fundamentally different architectural paradigm that renders traditional ablation impractical. The Graph Analyzer transforms raw system logs into structured provenance graphs and extracts community clusters that encapsulate semantic relationships essential for attack chain contextualization. The LLM Analyzer is explicitly designed to operate on these structured communities, not raw log sequences. Isolating the LLM Analyzer renders the system unable to perform its core functions of attack detection, investigation, and narrative generation, as it serves as the primary reasoning engine for the entire framework.

E.4. Summary of ablation results

The results demonstrate that each isolatable component contributes measurably to SHIELD's overall performance. The Deviation Analyzer serves as a critical preprocessing filter that maintains both accuracy and computational efficiency, while the Temporal Correlation Engine enables the system's fundamental capability to maintain attack context across temporal boundaries (see Table 7).

Table 5
Performance comparison of different models across datasets.

Model	Dataset	Confusion Matrix				Performance Metrics		
		TP	FN	FP	TN	Precision	Recall	F1-Score
GPT-4o	CADETS	07	05	21	681	25.0	58.3	35.0
	THEIA	06	03	00	294	100.0	66.7	80.0
	Arena	03	00	00	235	100.0	100.0	100.0
	BlindEagle	01	00	00	00	100.0	100.0	100.0
Qwen-2.5 32B	CADETS	07	05	17	685	29.2	58.3	38.9
	THEIA	06	03	04	290	60.0	66.7	63.2
	Arena	03	00	00	235	100.0	100.0	100.0
	BlindEagle	01	00	00	00	100.0	100.0	100.0
EXAONE-3.5 32B	CADETS	07	05	31	671	18.4	58.3	28.0
	THEIA	04	05	00	294	100.0	44.4	61.5
	Arena	02	01	00	235	100.0	67.0	80.2
	BlindEagle	01	00	00	00	100.0	100.0	100.0
LLama-3.1 8B	CADETS	06	06	48	654	11.1	50.0	18.2
	THEIA	05	04	03	291	62.5	55.6	58.8
	Arena	03	00	02	233	60.0	100.0	75.0
	BlindEagle	01	00	00	00	100.0	100.0	100.0

Table 6
Sensitivity Analysis of the alert threshold on the CADETS Dataset.

Threshold (δ)	Precision	Recall	F1-Score
0.70	0.84	0.93	0.88
0.75	1.00	0.93	0.96
0.80	1.00	0.93	0.96
0.85	1.00	0.93	0.96
0.90	1.00	0.72	0.84

Table 7
Ablation Study Results on CADETS dataset.

Component Removed	Precision	Recall	Runtime	Key Impact
None (Entire SHIELD Running)	1.00	0.94	~55 min	Baseline performance
Deviation Analyzer	0.87	0.94	~232 min	4–6x slower runtime, 13% precision loss
Temporal Correlation Engine	0.29	0.58	~55 min	Loss of attack correlation across time windows

Table 8
System runtime analysis.

Dataset	Avg. Graph Size (nodes, edges)	Deviation Analyzer	Graph Analyzer	LLM Analyzer	TCE Time
CADETS	(1151, 36235)	~1 s	~1 s	~30 s	~12 s
THEIA	(4127, 57247)	~1 s	~1 s	~20 s	~12 s
CLEARSCOPE	(1240, 48090)	~1 s	~1 s	~30 s	~13 s
OpTC	(1296, 54684)	~1 s	~1 s	~30 s	~15 s

Appendix F. Runtime analysis and bottleneck considerations

To address the practical deployment aspects of SHIELD, we conducted a comprehensive analysis of system resource overhead and potential bottlenecks. The SHIELD architecture is designed with computational efficiency in mind, where the Deviation Analyzer and Graph Analyzer modules operate on standard CPU resources without requiring specialized hardware. Overhead is only imposed when the logs for a community are passed, a step that requires either one on-prem GPU with approximately 48 GB of VRAM or a low-latency cloud endpoint.

We evaluated SHIELD's performance across four datasets using random 30-minute time windows to measure processing times for each component: The results are presented in Table 8.

We identified that the most significant bottleneck occurs during periods of high alert activity, where multiple simultaneous anomaly detections create GPU queuing delays. This can lead to processing backlogs that may impact real-time response capabilities.

While this bottleneck is manageable for typical operational deployment scenarios, extremely large organizations that experience high alert volumes may need to implement GPU clustering or cloud-based scaling solutions to ensure consistent real-time performance.

Appendix G. Analyst workload reduction analysis

To quantify the operational impact of SHIELD, we analyze the analyst workload in terms of events requiring manual review. For each detection method, the workload is calculated as the sum of true and false positives, representing the total number of alerts an analyst must investigate.

Fig. 6 presents this comparison across all four datasets. SHIELD consistently achieves the lowest analyst workload while maintaining high detection accuracy. This dramatic reduction in analyst workload directly addresses alert fatigue, a critical operational challenge in SOCs. By minimizing false positives while maintaining high recall, SHIELD enables security analysts to focus their efforts on genuine threats rather than investigating spurious alerts.

Data availability

Data will be made available on request.

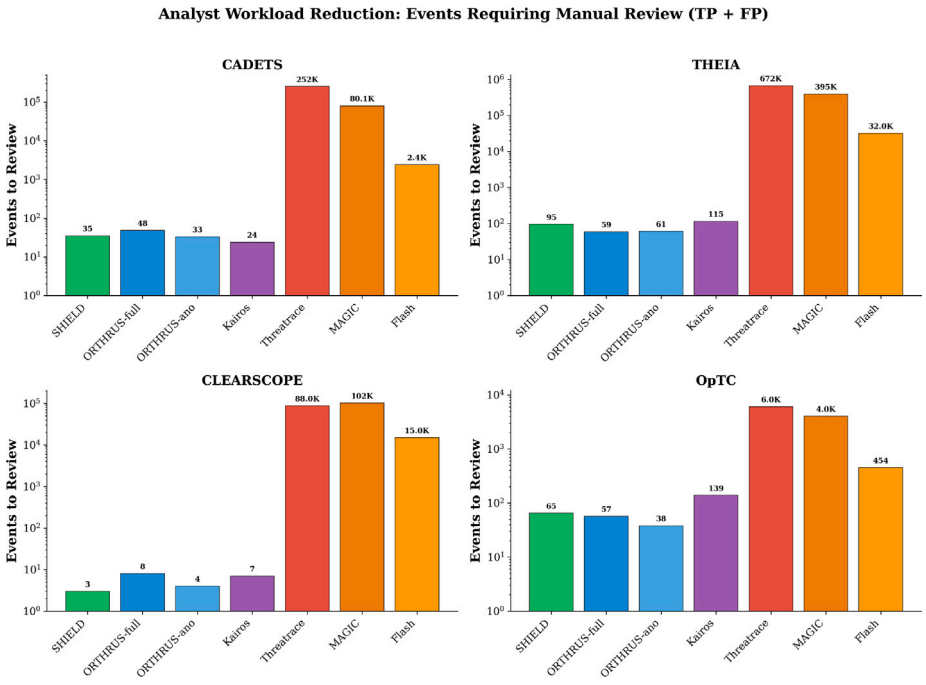


Fig. 6. Analyst workload comparison across datasets.

References

Ali, T., Kostakos, P., 2023. HuntGPT: Integrating machine learning-based anomaly detection and explainable AI with large language models (LLMs). URL https://arxiv.org/abs/2309.16021.

Alsaheel, A., Nan, Y., Ma, S., Yu, L., Walkup, G., Celik, Z.B., Zhang, X., Xu, D., 2021. {ATLAS}: A sequence-based learning approach for attack investigation. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 3005–3022.

Bilot, T., Jiang, B., Li, Z., El Madhoun, N., Al Agha, K., Zouaoui, A., Pasquier, T., 2025. Sometimes simpler is better: a comprehensive analysis of state-of-the-art provenance-based intrusion detection systems. In: Proceedings of the 34th USENIX Conference on Security Symposium. SEC '25, USENIX Association, USA.

Blondel, V.D., Guillaume, J.L., Lambiotte, R., Lefebvre, E., 2008. Fast unfolding of communities in large networks. J. Stat. Mech. Theory Exp. 2008 (10), P10008.

Chai, Y., Chen, X., Qiu, J., Du, L., Xiao, Y., Feng, Q., Ji, S., Tian, Z., 2024. Malfscil: A few-shot class-incremental learning approach for malware detection. IEEE Trans. Inf. Forensics Secur..

Chai, Y., Qiu, J., Yin, L., Zhang, L., Gupta, B.B., Tian, Z., 2022. From data and model levels: Improve the performance of few-shot malware classification. IEEE Trans. Netw. Serv. Manag. 19 (4), 4248–4261.

Cheng, Z., Lv, Q., Liang, J., Wang, Y., Sun, D., Pasquier, T., Han, X., 2023. Kairos: Practical intrusion detection and investigation using whole-system provenance. URL https://arxiv.org/abs/2308.05034.

Dong, F., Li, S., Jiang, P., Li, D., Wang, H., Huang, L., Xiao, X., Chen, J., Luo, X., Guo, Y., et al., 2023a. Are we there yet? an industrial viewpoint on provenance-based endpoint detection and response tools. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. pp. 2396–2410.

Dong, F., Wang, L., Nie, X., Shao, F., Wang, H., Li, D., Luo, X., Xiao, X., 2023b. {DISTDET}: A {Cost – Effective} distributed cyber threat detection system. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 6575–6592.

Du, M., Li, F., Zheng, G., Srikumar, V., 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 1285–1298.

Freitas, S., Kalajdjieski, J., Gharib, A., McCann, R., 2024. AI-driven guided response for security operation centers with microsoft copilot for security. URL https://arxiv.org/abs/2407.09017.

Guo, H., Yuan, S., Wu, X., 2021. Logbert: Log anomaly detection via bert. In: 2021 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–8.

Hagberg, A., Swart, P., S. Chult, D., 2008. Exploring Network Structure, Dynamics, and Function Using Networkx. Tech. rep., Los Alamos National Lab.(LANL), Los Alamos, NM (United States).

Han, X., Pasquier, T., Bates, A., Mickens, J., Seltzer, M., 2020. Unicorn: Runtime provenance-based detector for advanced persistent threats. arXiv preprint arXiv:2001.01525.

Han, X., Yuan, S., Trabelsi, M., 2023. LogGPT: Log anomaly detection via GPT. URL https://arxiv.org/abs/2309.14482.

Hassan, W.U., Bates, A., Marino, D., 2020. Tactical provenance analysis for endpoint detection and response systems. In: 2020 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1172–1189.

Hassan, W.U., Guo, S., Li, D., Chen, Z., Jee, K., Li, Z., Bates, A., 2019a. Nodoze: Combatting threat alert fatigue with automated provenance triage. In: Network and Distributed Systems Security Symposium.

Hassan, W.U., Guo, S., Li, D., Chen, Z., Jee, K., Li, Z., Bates, A., 2019b. Nodoze: Combatting threat alert fatigue with automated provenance triage. In: Proceedings 2019 Network and Distributed System Security Symposium. URL https://api.semanticscholar.org/CorpusID:56436113.

Hossain, M.N., Milajerdi, S.M., Wang, J., Eshete, B., Gjomemo, R., Sekar, R., Stoller, S., Venkatakrishnan, V., 2017. {SLEUTH}: Real-time attack scenario reconstruction from {COTS} audit data. In: 26th USENIX Security Symposium. USENIX Security 17, pp. 487–504.

Hossain, M.N., Sheikhi, S., Sekar, R., 2020. Combating dependence explosion in forensic analysis using alternative tag propagation semantics. In: 2020 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1139–1155.

Inam, M.A., Chen, Y., Goyal, A., Liu, J., Mink, J., Michael, N., Gaur, S., Bates, A., Hassan, W.U., 2023. Sok: History is a vast early warning system: Auditing the provenance of system intrusions. In: 2023 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 2620–2638.

Jia, Z., Xiong, Y., Nan, Y., Zhang, Y., Zhao, J., Wen, M., 2023. MAGIC: Detecting advanced persistent threats via masked graph representation learning. arXiv preprint arXiv:2310.09831.

Jiang, B., Bilot, T., El Madhoun, N., Al Agha, K., Zouaoui, A., Iqbal, S., Han, X., Pasquier, T., 2025. ORTHRUS: Achieving high quality of attribution in provenance-based intrusion detection systems. In: Security Symposium. USENIX Sec'25. USENIX.

Kurniawan, K., Ekelhart, A., Kiesling, E., Quirchmayr, G., Tjoa, A.M., 2022. KRYSTAL: Knowledge graph-based framework for tactical attack discovery in audit data. Comput. Secur. 121, 102828.

Lazarovitz, L., 2021. Deconstructing the solarwinds breach. Comput. Fraud Secur. 2021 (6), 17–19.

Lee, Y., Kim, J., Kang, P., 2021. LAnoBERT : System log anomaly detection based on BERT masked language model. CoRR abs/2111.09564, URL https://arxiv.org/abs/2111.09564.

Li, S., Dong, F., Xiao, X., Wang, H., Shao, F., Chen, J., Guo, Y., Chen, X., Li, D., 2024. NODLINK: An online system for fine-grained APT attack detection and investigation. In: Proceedings 2024 Network and Distributed System Security Symposium. In: NDSS 2024, Internet Society.

Liu, Y., Tao, S., Meng, W., Wang, J., Ma, W., Zhao, Y., Chen, Y., Yang, H., Jiang, Y., Chen, X., 2024. Interpretable online log analysis using large language models with prompt strategies. URL https://arxiv.org/abs/2308.07610.

Milajerdi, S.M., Gjomemo, R., Eshete, B., Sekar, R., Venkatakrishnan, V., 2019. Holmes: real-time apt detection through correlation of suspicious information flows. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1137–1152.

Pei, K., Gu, Z., Saltaformaggio, B., Ma, S., Wang, F., Zhang, Z., Si, L., Zhang, X., Xu, D., 2016. HERCULE: attack story reconstruction via community discovery on correlated log graph. In: Proceedings of the 32nd Annual Conference on Computer Security Applications. ACSAC '16, Association for Computing Machinery, New York, NY, USA, pp. 583–595.

Rehman, M.U., Ahmadi, H., Hassan, W.U., 2024. Flash: A comprehensive approach to intrusion detection via provenance graph representation learning. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 3552–3570.

van der Kouwe, E., Heiser, G., Andriesse, D., Bos, H., Giuffrida, C., 2019. SoK: Benchmarking flaws in systems security. In: 2019 IEEE European Symposium on Security and Privacy. EuroS&P, IEEE, pp. 310–325.

Wang, Q., Hassan, W.U., Li, D., Jee, K., Yu, X., Zou, K., Rhee, J., Chen, Z., Cheng, W., Gunter, C.A., et al., 2020. You are what you do: Hunting stealthy malware via data provenance analysis. In: NDSS.

Wang, S., Wang, Z., Zhou, T., Sun, H., Yin, X., Han, D., Zhang, H., Shi, X., Yang, J., 2022. Threatrace: Detecting and tracing host-based threats in node level through provenance graph learning. IEEE Trans. Inf. Forensics Secur. 17, 3972–3987.

Yang, F., Xu, J., Xiong, C., Li, Z., Zhang, K., 2023. {PROGRAPHER}: An anomaly detection system based on provenance graph embedding. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 4355–4372.

Yu, H., Li, A., Jiang, R., 2019. Needle in a haystack: attack detection from large-scale system audit. In: 2019 IEEE 19th International Conference on Communication Technology. ICCT, IEEE, pp. 1418–1426.

Zengy, J., Wang, X., Liu, J., Chen, Y., Liang, Z., Chua, T.S., Chua, Z.L., 2022. Shadewatcher: Recommendation-guided cyber threat analysis using system audit records. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 489–506.

Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., et al., 2019. Robust log-based anomaly detection on unstable log data. In: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 807–817.

Zhuang, H., Qin, Z., Hui, K., Wu, J., Yan, L., Wang, X., Bendersky, M., 2024. Beyond yes and no: Improving zero-shot LLM rankers via scoring fine-grained relevance labels. URL https://arxiv.org/abs/2310.14122.

Zipperle, M., Gottwalt, F., Chang, E., Dillon, T., 2022. Provenance-based intrusion detection systems: A survey. ACM Comput. Surv. 55 (7), 1–36.

Zuo, F., Rhee, J., Choe, Y.R., 2025. Knowledge transfer from LLMs to provenance analysis: A semantic-augmented method for APT detection. URL https://arxiv.org/abs/2503.18316.